A Genetic Algorithm for the Orienteering Problem

M. Fatih Tasgetiren
Department of Industrial and Systems Engineering
Auburn University
207 Dunstan Hall
Auburn, AL 36849-5346 USA
Email: mfatih@eng.auburn.edu

Alice E. Smith
Department of Industrial and Systems Engineering
Auburn University
207 Dunstan Hall
Auburn, AL 36849-5346 USA
Email: aesmith@eng.auburn.edu

Abstract: This paper presents a genetic algorithm to solve the orienteering problem, which is concerned with finding a path between a given set of control points, among which a start and an end point are specified, so as to maximize the total score collected subject to a prescribed time constraint. Employing three sets of test problems from the literature, the performance of the genetic algorithm is evaluated against problem specific heuristics and an artificial neural network.

1. Introduction

Given a set of control points with associated scores along with the start and end points, the orienteering problem (OP) deals with finding a path between the start and end points in order to maximize the total score subject to a given time budget, denoted by TMAX. Because of the limited time, tours may not include all points. It should also be noted that the OP is equivalent to the Traveling Salesman Problem (TSP) when the time is relaxed just enough to cover all points and start and end points are not specified.

To model the orienteering problem, we denote V as the set of control points and E as the set of edges between points in V. Then, the complete graph can be defined by $G = \{V, E\}$. Each control point, i, in V has an associated score $S_i \geq 0$ whereas the start point 1 and the end point N have no scores. The distance, d_{ij}, between points i and point j is the nonnegative cost for each edge in E or the cost of traveling between point i and point j. So, the objective is to find a path from start point 1 to end point N through a subset of control points so that the total score collected from the visited points will be maximized without violating the given time constraint. The OP is NP-hard and has applications in vehicle routing and production scheduling, as discussed in Golden et al. [5] and Keller [8].

Most research concerning the OP falls into two categories: heuristic approaches or exact solution methods. Tsiligirides [15], Golden et al. [6,7], Keller [8], Ramesh and Brown [12], Sokkappa [14] and Chao et al. [1] developed heuristic algorithms to solve the OP problem and compared results using three sets of test problems first provided by Tisiligirides. On the other hand, exact solution methods have been presented by Laporte and Martello [9], Sokkapa [14], Ramesh et al. [13], Pillai [11], and Fishetti et al [3]. In addition, Leifer and Rosenwein [10] determined upper bounds on the value of the objective function for the test problems provided by Tsiligirides.

2. Genetic Algorithm

Figure 1 shows the overall scheme of the genetic algorithm (GA) for the OP, and this is detailed in the subsections of this section. First, the initial population of size λ is constructed probabilistically based on the distance and TMAX information for each problem. At each generation, two parents are determined by tournament selection of size 2 and random selection, respectively, to produce an offspring through order-based crossover. This process is conducted in a loop until μ offspring (pop size*crossover probability) are produced. Hence, the size of the population is increased to $(\lambda+\mu)$ at the end of each generation. Then, local search mutation consisting of add_a_point, omit_a_point and replace_a_point routines is applied to individuals selected randomly from the offspring. For the population of the next generation, tournament selection of size 2 is used to establish the population, again among $(\lambda+\mu)$ individuals, thus maintaining a λ size

0-7803-6375-2/00/$10.00 ©2000 IEEE.

population. This procedure is repeated until the stopping criterion is achieved.

Figure 1. Pseudo code for the genetic algorithm.

Initialize population P of size λ
Evaluate λ *individuals in P*
While not termination do {
*Select 2** μ *individuals from P*
Crossover individuals to produce μ *offspring*
Mutate some individuals in μ
Add μ *offspring to* λ *individuals in P*
Evaluate $(\lambda+\mu)$ *individuals in P*
Select λ *individuals from* $(\lambda+\mu)$ *individuals in P }*
End While
End Algorithm

2.1 Permutation Representation

In an OP tour, points are listed in the same order they are visited by keeping the start and end points the same in every chromosome. This is also called a *path* or *order* representation. An example is given below. 0's in the list mean that some cities in the list have been omitted from this tour. All tours must begin at point 1 and end at point 32, and this particular tour visits points 10, 15, 19, 17, 14, 25, 27, 26, 31 and 30 in that order.

1,0,10,0, 0, 0, 0, 0, 0, 15, 19, 17, 0, 0, 0,14, 0, 0, 0, 0, 0, 0, 25,27, 0,0, 0, 26, 31,0,30. 32

2.2. Initial Population

In order to construct the initial population, we used a point omission probability based on the TMAX constraint and distance information for each problem as shown in Figure 2. This is done so that the initial population generally follows the TMAX constraint, i.e. not too few and not too many points are included in each tour. A list of the N points is generated in random order. Then, each point is assigned a random number [1,0] and, if this is less than the point omission probability, the point is omitted from the tour by simply putting zero onto it, otherwise it is included in the tour.

Figure 2. Point omission probability

Define TMAX
Define maxloop
Set count=0 and loop=0;
Do{
Produce a random number, R_N*, between [1,N]*
Produce a list, R_L*, between [1,N] randomly*
Generate a sub list, R_S*, by taking the first* R_N *part of the random list* R_L
Compute the total distance, d_{RS}*, of the sub list* R_S
If d_{RS} *<=TMAX, then count=count+1*
loop=loop+1
}while (loop<=maxloop)
*point omission probability=*1-*count/maxloop*

2.3 Order Based Crossover Operator

Order-based crossover [4] is used so that a repair algorithm is not needed. In order-based crossover, a sub list is selected from parent1 and copied into the corresponding positions of the offspring. Then, the points in the sub list are deleted from parent 2. The resulting points in parent 2 are inserted into the offspring wherever necessary to complete the offspring. The following is an example (remember that points 1 and 32 are fixed).

jcross1=10 jcross2=26
Parent1:
1,0,9,0,0,0,0,0,0,14,18,16,0,0,0,13,0,0,0,0,0,0,24 26,0,0,0,25,30,0,29,32

Parent2:
1,0,2,10,8,0,0,0,7,0,0,0,0,17,13,0,0,0,21,0,0,0, 0,0,0,0,0,0,27,0,30,32

- The sub list between 10 and 26 from parent 1 is taken and copied into the corresponding positions in the offspring. Then these points in the sub list are deleted from parent 2.
- The resulting sequence is reordered in a way that the 0's will be at the end of the list. So the resulting list is:
  2,10,8,7,17,21,27,30,0,0,0,0,0,0,0,0,0,0,0,0

- The resulting sequence is copied into the offspring in a way that first the left side of the offspring is filled in and then the right side is filled in.

Offspring:
1,2,10,8,7,17,21,27,30,0,18,16,0,0,0,13,0,0,0,0,0,
0,24,26,0,0,0,0,0,0,0,32

2.4 Local Search Mutation

For mutation, we employed local search to enrich and diversify the population by means of add_a_point, omit_a_point and replace_a_point routines, which is applied to only newly created offspring. Through this local search, solutions near the feasibility border (i.e., where TMAX is active) are improved while solutions far from the feasibility border are worsened. An offspring is randomly selected, and the add_a_point routine is applied 10 times. The best one of these is replace into the population of μ offspring. This procedure is then repeated for the omit_a_point procedure, and then finally for the replace_a_point procedure. This entire procedure of three local search mechanisms is repeated to generate the specified number of mutants (i.e., population size times mutation probability).

Add a point:
Offspring:
1,0,9,0,0,0,0,0,0,14,18,16,0,0,0,13,0,0,0,0,0,0,0,24,
26,0,0,0,25,30,0,29,32

- Find a point that is not in the offspring, say 8.
- Add it to the offspring in a position selected randomly

Modified Offspring:
1,0,9,0,0,0,8,0,0,14,18,16,0,0,0,13,0,0,0,0,0,0,0,24
26,0,0,0,25,30,0,29,32

- Evaluate the offspring.
- Repeat ten times.
- Select the best and replace into offspring population.

Omit a point:
Offspring:
1,0,9,0,0,0,0,0,0,14,18,16,0,0,0,13,0,0,0,0,0,0,0,24
26,0,0,0,25,30,0,29,32

- Find a point that is in the offspring, say 18.
- Delete it from the offspring.

Modified Offspring:
1,0,9,0,0,0,8,0,0,14,0,16,0,0,0,13,0,0,0,0,0,0,0,24,
26,0,0,0,25,30,0,29,32

- Evaluate the offspring.
- Repeat ten times.
- Select the best and replace into offspring population.

Replace a point:
Offspring:
1,0,9,0,0,0,0,0,0,0,14,18,16,0,0,0,13,0,0,0,0,0,0,0,24
26,0,0,0,25,30,0,29,32

- Find a point that is not in the offspring, say 21.
- Select a point randomly in the offspring, say 14, and replace it with the randomly selected point not in the offspring, 21.

Modified Offspring:
1,0,9,0,0,0,8,0,0,21,18,16,0,0,0,13,0,0,0,0,0,0,0,24
26,0,0,0,25,30,0,29,32

- Evaluate the offspring.
- Repeat ten times.
- Select the best and replace into the offspring population.

2.5 Evaluation

Since the OP is a constrained problem and search can benefit from considering some infeasible solutions [2], a penalty function for infeasible solutions is used:

$$penalizedfitness = fitness * \left(1 - \frac{tdistance - TMAX}{tdistance}\right)^{\alpha}$$

where *tdistance* is the total distance of the individual and α is the severity parameter of the penalty function. Thus, solutions are penalized according to their distance from feasibility.

2.6 Parameters

The GA parameters have been determined experimentally: population size = 100, crossover probability = 0.70, mutation probability = 0.40 and α = 4. The GA terminated after the best feasible solution remains unchanged for δ generations. δ varies with TMAX since a larger TMAX allows a longer string of visited points and needs a larger δ. δ increases from 5 to 100 as TMAX increases.

3. Computational Results

The GA is coded in Borland C++ and implemented on a Dell 450 PC. All computations are conducted using real precision without rounding or truncating values. The precision of the final solution length has been rounded to one decimal point as in Chao [1]. We examined

three sets of test problems with 32, 21 and 33 points provided by Tsiligirides [15] as well as the corrected problem set 1 by Chao [1].

During the course of these investigations, we found an error in previously published results. Chao found distances of 88.23 and 87.74 for two solutions provided by Tsiligirides. We analyzed the data to calculate the distances for these solutions and found them to be 88.65 and 88.16, respectively, differences of 0.42 each. We also analyzed the sequences and their distance information provided by Keller [8]. In the problem with TMAX=15 in the first data set, using the same sequence as in Keller we computed a distance of 14.70 instead of their 14.28, a difference of 0.42 again. Reviewing this data, we conclude that the distance information between points 19 and 20 has been undercalculated in previous work by 0.42.

The GA is applied on 67 problems from four test sets and compared to Tsiligirides's stochastic algorithm T [15], Chao's heuristic C [1] and an artificial neural network NN [16]. Computational results are given in tables 1-4 and the GA distance and sequences for the best solutions to all test problems are given in the Appendix.

From tables 1-4, it is clear that the GA outperforms Tsiligirides's stochastic algorithm. In comparison to the ANN, the GA produces the same or better results in 66 of 67 cases. In comparison to Chao's heuristic, the GA produces the same results in 61 of 67 cases. In one case, Chao's heuristic gave better results while in five cases, the GA produces better results. However, it should be noted that in two of the five cases (TMAX=30 and TMAX=40) in the second data set the results depend on rounding the time to one decimal point, as in Chao. We also include alternative solutions for these two cases without rounding to one decimal point, in which case the first reported solutions slightly violate TMAX. CPU times are comparable to Chao's heuristic.

4. Conclusions

To our knowledge, this is the first reported application of GA to the OP. It is seen that the GA performed very well across a wide variety of problem instances and degrees of constraint.

Table 1. Comparison of results on data set 1.

TMAX	T	C	NN	GA
5	10	10	10	10
10	15	15	15	15
15	45	45	45	45
20	65	65	65	65
25	90	90	90	90
30	110	110	110	**115**
35	135	135	135	135
40	150	155	155	155
46	175	175	175	175
50	190	190	190	190
55	205	205	205	205
60	220	225	225	225
65	240	240	240	240
70	255	260	260	260
73	260	265	265	265
75	270	270	270	270
80	275	280	280	280
85	280	285	285	285

Table 2. Comparison of results on data set 2.

TMAX	T	C	NN	GA
15	120	120	120	120
20	190	200	200	200
23	205	210	205	210
25	230	230	230	230
27	230	230	230	230
30*	250	265	265	**275**
30	250	265	265	**265**
32	275	300	300	300
35	315	320	320	320
38	355	360	360	360
40*	395	395	395	**400**
40	395	395	395	**395**
45	430	450	450	450

*Rounded to one decimal point.

Table 3. Comparison of results on data set 3.

TMAX	T	C	NN	GA
15	100	170	170	170
20	140	200	200	200
25	190	260	260	260
30	240	320	320	320
35	290	390	390	390
40	330	430	420	430
45	370	470	470	470
50	410	520	520	520
55	450	550	550	550
60	500	580	580	580
65	530	610	610	610
70	560	640	640	640
75	590	670	670	670
80	640	710	700	710
85	670	740	740	740
90	690	770	770	770
95	720	790	790	790
100	760	800	800	800
105	770	800	800	800
110	790	800	800	800

Table 4. Comparison of results on data set 4.

TMAX	T	C	NN	GA
5	10	10	10	10
10	15	15	15	15
15	45	45	45	45
20	65	65	65	65
25	90	90	90	90
30	110	110	110	**115**
35	135	135	130	135
40	150	155	155	155
46	175	175	175	175
50	190	190	190	190
55	205	205	205	205
60	220	220	220	**225**
65	240	240	240	240
70	255	260	260	260
73	260	265	265	265
75	270	**275**	**275**	270
80	275	280	280	280
85	280	285	285	285

Acknowledgment

The first author wishes to thank Sakarya University in Turkey for its support of this research in the U.S.A and is grateful to the Rector, Prof. Dr. Ismail Calli, for his endless support and vision.

Bibliography

[1] Chao I-M., Golden B.L., and Wasil E. A., 1996, A fast and effective heuristic for the orienteering problem, *European Journal of Operational of Operational Research*, No. 88, 475-489.

[2] Coit D.W. and Smith A. E., 1996, Penalty guided genetic search for reliability design optimization, *Computers and Industrial Engineering*, Vol. 30, No. 4, 895-904.

[3] Fischetti M., Gonzales J.S., and Toth P., 1998, Solving the orienteering problem through branch-and-cut, *INFORMS Journal on Computing*, Vol.10, No.2, 133-148.

[4] Gen M. and Cheng R., 1997, Genetic algorithms & engineering design, *John Wiley & Sons*, Wiley Series in Engineering Design and Automation.

[5] Golden B. L., Assad A. and Dahl R., 1984, Analysis of a large-scale vehicle routing problem with inventory component, *Large Scale Systems*, No. 7, 181-190

[6] Golden B.L., Levy L., and Vohra R., 1987, The orienteering problem, *Naval Research Logistics*, No. 34, 307-318

[7] Golden B.L., Wang Q, and Liu L., 1988, A multifaceted heuristic for the orienteering problem, *Naval Research Logistics*, No. 35, 359-366

[8] Keller P., 1989, Algorithms to solve the orienteering problem: a comparison, *European Journal of Operational of Operational Research*, No. 41, 224-231.

[9] Laporte G. and Martello S., 1990 The selective traveling salesman problem, *Discrete Applied Mathematics*, No. 26, 193-207

[10] Leifer A.C, and Rosenwein M.S., Strong linear programming relaxations for the orienteering problem, *European Journal of Operational Research*, No. 73, 517-523.

[11] Pillai R.S., 1992, The traveling salesman subset-tour problem with one additional constraint (TSSP+1), *Ph.D. Dissertation*, The University of Tennessee, Knoxville, TN.

[12] Ramesh R., and Brown K. M., 1991, An efficient four-phase heuristic for the generalized orienteering problem, *Computers Ops Res.* Vol. 18. No.2, 151-165.

[13] Ramesh R., Yoon Y-S., and Karwan M. H., 1992, An optimal algorithm for the orienteering tour problem, *ORSA Journal of Computing*, Vol.4, No.2, 155-165.

[14] Sokkappa P.R., 1990 The cost-constrained traveling salesman problem, *Ph.D. Dissertation*, The University of California, Livermore, CA.

[15] Tsiligirides T., 1984, Heuristic Methods Applied to Orienteering. *Journal of Operational Research Society*, Vol. 35, No. 9, 797-809

[16] Wang Q., Sun X., Golden B. L., and Jia J., 1995, Using artificial neural networks to solve the orienteering problem, *Annals of Operations Research*, No. 61, 111-120.

Appendix

Table A1. Sequence Found by the GA for Problem Set 1.

TMAX	d	Sequence
5	4.14	1 28 32
10	6.87	1 28 18 32
15	14.70	1 27 31 26 20 19 32
20	18.99	1 19 20 21 22 26 31 27 32
25	23.88	1 27 31 26 25 23 22 21 20 19 32
30	29.94	1 28 27 31 26 25 24 23 22 21 20 19 32
35	34.08	1 28 27 31 26 25 23 22 21 12 11 10 8 9 13 32
40	38.97	1 28 27 31 26 25 23 22 21 12 11 10 8 2 3 7 6 32
46	45.63	1 28 27 31 26 25 24 23 22 21 20 12 11 10 8 2 3 7 6 32
50	49.19	1 28 27 31 26 25 24 23 22 21 12 11 10 9 8 2 3 7 32
55	54.80	1 28 27 31 26 25 23 22 21 12 11 10 8 2 3 7 6 5 4 14 15 18 32
60	59.89	1 27 31 26 25 23 22 21 12 11 10 8 2 3 7 6 5 4 14 15 16 17 28 32
65	63.88	1 28 17 16 15 14 4 5 6 7 3 2 8 10 11 12 21 22 23 24 25 26 31 27 32
70	69.56	1 28 29 17 16 15 14 4 5 6 7 3 2 8 10 11 12 21 22 23 24 25 26 31 27 20 19 32
73	72.16	1 28 29 17 16 15 14 4 5 6 7 3 2 8 9 10 11 12 21 22 23 24 25 26 31 27 19 20 32
75	74.22	1 19 20 27 31 26 25 24 23 22 21 12 11 10 9 8 2 3 7 6 5 2 14 15 16 17 29 28 18 32
80	79.30	1 28 29 17 16 15 14 4 5 6 7 3 2 13 9 8 10 11 12 21 22 23 24 25 30 31 26 27 20 19 32
85	82.89	1 18 28 29 17 16 15 14 4 5 6 13 7 3 2 8 9 10 11 12 21 22 23 24 25 30 31 26 27 20 19 32

Table A2. Sequence Found by the GA for Problem Set 2.

TMAX	d	Sequence
15	14.56	1 5 6 7 12 11 10 13 14 21
20	19.88	1 12 7 6 5 3 2 8 9 10 11 13 14 21
23	22.65	1 7 6 5 4 3 2 8 9 10 11 14 21
25	24.13	1 12 7 6 5 4 3 2 8 9 10 11 13 14 21
27	24.13	1 12 7 6 5 4 3 2 8 9 10 11 13 14 21
30*	30.00	1 7 6 5 3 2 8 17 16 15 9 10 11 13 21
30	29.85	1 7 6 2 8 17 16 15 9 10 11 13 14 21
32	31.63	1 7 6 5 3 2 8 17 16 15 9 10 11 13 14 21
35	34.51	1 7 6 5 3 4 20 19 18 17 9 10 11 13 14 21
38	37.84	1 7 6 5 2 3 4 20 19 18 17 8 9 10 11 13 14 21
40*	40.00	1 7 6 5 3 4 20 19 18 16 15 17 9 10 11 13 14 21
40	39.78	1 7 6 5 3 4 20 19 18 16 15 17 8 9 10 11 13 21
45	44.44	1 12 7 6 5 2 3 4 20 19 18 16 15 17 8 9 10 11 13 14 21

* Rounded to one decimal place.

Table A3. Sequence Found by the GA for Problem Set 3.

TMAX	d	Sequence
15	14.47	1 24 22 7 5 14 4 27 23 33
20	19.79	1 24 22 7 5 28 14 4 3 27 23 33
25	24.46	1 24 22 7 5 14 4 20 13 3 23 33
30	29.47	1 24 22 7 5 28 14 4 20 17 13 3 27 23 33
35	34.79	1 24 22 7 5 28 14 4 20 17 16 15 13 3 23 33
40	38.95	1 24 22 7 5 28 14 4 3 20 17 16 15 13 6 2 32 33
45	44.66	1 24 22 7 5 28 14 4 3 13 20 17 16 15 6 2 8 29 26 33
50	48.94	1 24 22 7 5 28 14 4 20 17 16 15 13 3 6 2 8 31 12 29 26 33
55	54.60	1 24 22 7 5 28 4 14 20 17 16 15 13 3 6 2 8 31 12 29 30 26 32 33
60	59.72	1 24 22 7 5 28 14 4 3 13 20 17 21 16 15 6 2 8 31 12 29 26 23 33
65	64.31	1 24 22 7 5 27 14 4 28 20 17 21 16 15 13 3 6 2 8 31 12 29 30 26 32 33
70	69.57	1 24 23 27 22 7 5 28 14 4 20 17 21 16 15 13 3 6 2 8 31 12 11 30 29 26 33
75	74.64	1 24 22 7 5 28 14 4 20 17 21 16 15 13 3 6 2 8 31 12 11 10 30 29 26 32 23 33
80	*80.00*	1 24 25 9 10 18 19 11 29 12 31 8 2 6 3 13 15 16 17 20 4 14 28 5 7 22 27 23 33
85	85.00	1 24 22 7 5 28 14 4 20 17 16 15 13 3 6 2 8 31 12 11 19 18 10 9 30 29 26 32 27 23 33
90	89.82	1 24 22 7 5 28 14 4 20 17 21 16 15 13 3 6 2 8 31 12 29 26 30 11 19 18 10 9 25 33
95	93.96	1 27 23 24 22 7 5 28 14 4 20 17 21 16 15 13 3 6 2 8 31 12 29 11 19 18 10 9 30 26 32 33
100	98.28	1 24 23 27 22 7 5 14 4 28 20 17 16 21 15 13 3 6 2 8 31 12 11 19 18 10 9 25 30 29 26 32 33
105	99.36	1 24 23 27 22 7 5 28 14 4 13 15 16 21 17 20 3 6 2 32 26 30 29 8 31 12 11 19 18 10 9 25 33
110	105.85	1 24 7 5 22 27 14 28 20 17 21 16 15 13 4 3 6 2 8 31 12 30 25 9 10 18 19 11 29 26 32 23 33

Table A4. Sequence Found by the GA for Problem Set 4.

TMAX	d	Sequence
5	4.14	1 28 32
10	6.87	1 28 18 32
15	14.70	1 27 31 26 20 19 32
20	18.99	1 19 20 21 22 26 31 27 32
25	23.88	1 27 31 26 25 23 22 21 20 19 32
30	29.94	1 28 27 31 26 25 24 23 22 21 20 19 32
35	34.08	1 28 27 31 26 25 23 22 21 12 11 10 8 9 13 32
40	38.97	1 28 27 31 26 25 23 22 21 12 11 10 8 2 3 7 6 32
46	45.75	1 13 6 7 3 2 8 10 11 12 21 22 23 24 25 26 31 27 28 32
50	49.79	1 28 27 31 26 25 24 23 22 21 12 11 10 8 2 3 4 5 7 6 32
55	54.80	1 28 27 31 26 25 23 22 21 12 11 10 8 2 3 7 6 5 4 14 15 18 32
60	59.89	1 27 31 26 25 23 22 21 12 11 10 8 2 3 7 6 5 4 14 15 16 17 28 32
65	63.88	1 28 17 16 15 14 4 5 6 7 3 2 8 10 11 12 21 22 23 24 25 26 31 27 32
70	69.56	1 28 29 17 16 15 14 4 5 6 7 3 2 8 10 11 12 21 22 23 24 25 26 31 27 20 19 32
73	72.16	1 28 29 17 16 15 14 4 5 6 7 3 2 8 9 10 11 12 21 22 23 24 25 26 31 27 19 20 32
75	74.99	1 28 30 29 17 16 15 14 4 5 6 7 3 2 8 9 10 11 12 20 21 22 23 24 25 26 31 27 32
80	78.83	1 18 28 30 29 17 16 15 14 4 5 6 7 3 2 8 9 10 11 12 21 22 23 24 25 26 31 27 20 19 32
85	84.12	1 18 28 30 29 17 16 15 14 4 5 6 13 7 3 2 8 9 10 11 12 19 20 21 22 23 24 25 26 31 27 32